BenchScout: Compressing Cloud-Serving Evaluation with Execution-Anchored Sparse Probes

Anonymous Author(s)

Abstract

Benchmarking is central to cloud-infrastructure operations, where teams compare instances, detect regressions, and tune services. Yet comprehensive benchmark suites can take minutes or hours to run, making repeated evaluation costly in both time and shared compute resources. We observe that this breadth also creates redundancy: across benchmark components and Web workload levels, a small number of informative target-system measurements can anchor useful estimates of the complete profile. We present BenchScout, which uses host context to automatically select short, duration-bounded probes, executes them on the target system, and reconstructs unmeasured results from the observed anchors. We evaluate it on UnixBench and PTS-CPU across five Linux hosts, PTS-GPU on two hosts, and CloudSuite Web Serving. In run-held-out UnixBench evaluation, BenchScout achieves 0.21% median suite-score error while reducing the estimated context-plus-probe budget from roughly 1,690 s to 18.1 s (about 99%). On CloudSuite Web Serving, three measured load levels reduce cold-start throughput-curve MAPE from 39.16% for history-only estimation to 7.21% with adaptive selection and interpolation. These results show that automated, execution-anchored measurement can substantially reduce evaluation time while retaining accurate benchmark and Web performance profiles.

CCS Concepts

• **Software and its engineering** → **Software performance**; Software reliability; • **Computing methodologies** → *Machine learning*; • **Computer systems organization** → Dependable and fault-tolerant systems and networks.

Keywords

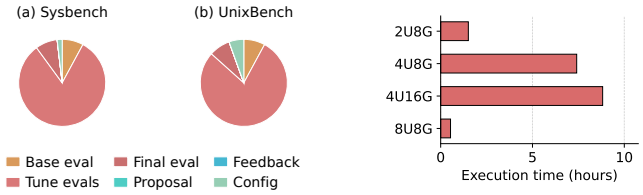
Web Infrastructure, HTTP Serving, System Benchmarking, Benchmark Suites, Performance Evaluation, Machine Learning, Sparse Measurement, Time-bounded Probing

ACM Reference Format:

Anonymous Author(s). 2027. BenchScout: Compressing Cloud-Serving Evaluation with Execution-Anchored Sparse Probes. In *Proceedings of The Web Conference 2027 (WWW '27), May 10–14, 2027, Dublin, Ireland*. ACM, New York, NY, USA, 16 pages.

1 Introduction

Benchmark suites [23, 30, 34] characterize modern computing platforms by running standardized workloads across processors, memory, storage, system software, and accelerators to produce a repeatable performance profile beyond any single measurement. They are widely used in performance-critical infrastructure workflows, including cloud instance comparison [1, 4, 17, 25, 26, 28], monitoring [5], and performance autotuning [7, 29, 35, 40]. However, the same breadth that makes benchmark suites useful also makes them costly to execute in full. As Figure 1 illustrates, benchmark evaluation can dominate an auto-tuning task (Figure 1a) and incur substantial wall-clock overhead across machine configurations (Figure 1b). Comprehensive suites combine heterogeneous workloads, each consuming time and competing for CPU, memory, storage, or accelerator resources. This cost compounds when measurements are repeated across machines, instance types, software revisions, and configurations.



(a) Time breakdown of two representative KeenTune tasks.

(b) Full-suite PTS-CPU execution time across machines.

Figure 1: Execution-time cost of benchmark-driven tuning and full-suite benchmark evaluation.

Repeated evaluation amplifies this burden: autotuning revisits configurations, monitoring checks for regressions, and cloud users compare many candidate instances. BenchScout reduces this cost by automatically choosing a small number of informative measurements on the target system, bounding their duration, and reconstructing the remaining profile from those execution anchors. Unlike a fixed subset, the selected probes can respond to the host's measured context; unlike execution-free prediction, the estimates remain grounded in current target-system measurements.

We make three contributions:

- **Profile-preserving measurement compression.** We formulate evaluation as selecting target-system coordinates under a duration budget, then reconstructing the unmeasured components or load points so the output retains the diagnostic structure of a full profile.
- **A host-local suite pipeline.** BenchScout combines context-conditioned Top-K routing, time-bounded component probes, and masked reconstruction to estimate suite components and an aggregate target while preserving the benchmark's published semantics.
- **Cross-workload accuracy–cost evidence.** Run-held-out UnixBench/PTS results and session-held-out CloudSuite results quantify profile fidelity alongside probe or phase cost. The evaluation spans component suites and multi-tier Web serving, with history, fixed, and adaptive controls that reveal how sparse measurement performs under stable and cold-start conditions.

These settings use different coordinates, split units, and estimators: run-held-out benchmark-suite profiles and session-held-out Web throughput curves. The Web study is a coordinate-specific instantiation of the same router–probe–reconstructor objective; its selector

and reconstructor are specialized for offered-load curves, so we test role-level unity without claiming parameter-level transfer from UnixBench.

2 Background and Motivation

2.1 Benchmark Suites and Web Profiles

Benchmark suites provide reproducible and quantifiable measurements for comparing system performance across machines, configurations, and software versions. Rather than relying on a single workload, a suite decomposes system performance into multiple components, each targeting a distinct workload pattern across computation, memory, storage, networking, process control, and interprocess communication. We denote a suite as $\mathcal{B} = \{b_1, b_2, \ldots, b_N\}$, where b_i is the i-th component. For a system state s, the full result of component b_i is $y_i = f_i(s, T_i, \epsilon_i)$, where T_i is the execution budget required by the original benchmark protocol and ϵ_i denotes measurement noise; a full-suite run yields the result vector $\mathbf{y} = [y_1, y_2, \ldots, y_N]$ and, when defined by the benchmark framework, an aggregate score $Y = A(\mathbf{y})$. Section 3.5 instantiates $A(\cdot)$ for UnixBench and PTS.

The cost of full-suite benchmarking comes from two sources: the number of components and the duration of each component. Formally,

$$C_{\text{full}} = \sum_{i=1}^{N} C_i(T_i). \tag{1}$$

This cost can be substantial for suites with many components or long measurement intervals. Practitioners invoke suites for two reasons with different credibility needs: repeated *accelerated formal* evaluation that must approximate the same Y, and *fast portraits* when a full run is infeasible. However, not all observations are equally informative: components stress overlapping subsystems (Section 2.2), and prefix executions often expose the same rate trend as run-to-completion. Our goal is therefore to estimate $(\mathbf{y}, Y)$ from sparse, time-bounded probes while keeping real measurements in the loop: component estimates are either directly probed on the target host or reconstructed from probe-imputed partial observations. The aggregate target is the published UnixBench index for UnixBench and a paper-defined log-mean proxy for PTS.

The same representation applies to a Web service: for configuration c and offered-load levels $\lambda_1, \ldots, \lambda_L$, its throughput profile is $\mathbf{q}_c = [q_c(\lambda_1), \ldots, q_c(\lambda_L)]$. The sparse-profile task selects a subset of coordinates under a preset duration cap for each target measurement, then estimates the remaining entries. A suite selects components; a Web workload selects offered-load levels. The endpoint and estimator are workload-specific even though the budgeted reconstruction problem is shared.

2.2 Motivation

Motivation. Suite components often share subsystem bottlenecks, while iterative rate tests expose useful signal before their full interval ends. We therefore test small fixed-duration selections against full references and matched history/fixed-subset controls. Optional PMU/eBPF telemetry is recorded with availability flags so missing

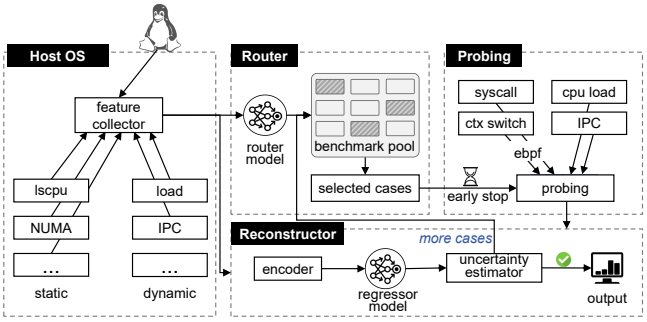


Figure 2: Overview of the BenchScout architecture. (1) Host OS: collector aggregates static and dynamic metrics into x. (2) Router: scores experts and selects Top-K. (3) Probing: category micro-workloads (or real subtests) run under a fixed budget and emit p. (4) Reconstructor: multi-output regressor predicts $\hat{\mathbf{y}}$ and $\hat{Y}$.

counters do not change the feature schema (Section 3.2); the detailed correlation and convergence diagnostics are reported in the supplementary artifact.

3 Methods

3.1 Overview

BenchScout approximates a complete benchmark run by observing only a small selected subset (Figure 2). Given the current system state, it collects context $\mathbf{x}$, routes Top-K subtests, runs fixed-duration probes on the selected components, and reconstructs the full official profile (Sections 3.3–3.5). The pipeline is *single-pass*: budgets (K, probe duration) are fixed at deployment; there is no adaptive re-probing loop.

Offline training and online protocol. **Offline**, each host h collects M complete runs $\{(\mathbf{x}_i, \mathbf{y}_i, Y_i)\}_{i=1}^{M}$ and trains a router f_θ, probe estimators f_{e_j}, and a reconstructor g_ϕ, persisting them as a host-local checkpoint bundle with no cross-host weight sharing. **Online**, BenchScout runs four deterministic stages: **Profile** $\mathbf{x}$ (3 s warmup, 0.5 s `/proc` sampling, perf PMU when permitted); **Route** the Top-K experts via f_θ; **Probe** each selected expert for a fixed T_{probe} (default 4 s) to obtain $\hat{y}_j$; and **Reconstruct** the sparse tuple $Z = [\mathbf{x}, m_j, \hat{y}_j, \tau_j]$ into $\hat{\mathbf{y}}, \hat{Y} = g_\phi(Z)$. The budget estimate is $T_{\mathbf{x}} + T_{\text{selected probes}}$ (about 18 s in the current hybrid artifacts, including a 3 s context estimate and roughly 15 s of selected probes) versus C_{full} (Equation 1); unlike execution-free predictors [36, 41], each prediction uses bounded on-host probes and per-host training. PTS suite comparisons use the proxy defined in Section 3.5.

Complexity and design invariants. The online cost is dominated by context collection and probing rather than model inference:

$$C_{\text{partial}} \approx T_{\mathbf{x}} + K \cdot T_{\text{probe}} + T_{\text{infer}}, \tag{2}$$

where T_{infer} is a sub-second tree/MLP forward pass and $T_{\mathbf{x}}$ is fixed regardless of suite cardinality N, whereas C_{full} scales with every component (Equation 1). This is a budget estimate; end-to-end wall time and interference must be measured for each deployment environment before operational use.

3.2 Feature Collection and Engineering

For run i, the pre-run context $\mathbf{x}_i = [\mathbf{x}_i^{\text{static}}, \mathbf{x}_i^{\text{dynamic}}]$ combines stable host descriptors (CPU topology, NUMA layout, memory, OS/compiler versions) with transient pressure signals (utilization, iowait, load, runnable tasks, and perf/`/proc` counters). Each probe of component b_j yields a fixed-order prefix vector $\mathbf{p}_{i,j}$ containing progress and elapsed time, optional eBPF scheduling/syscall rates, `/proc` resource summaries, workload category, probe mode, and failure flags (Appendix Table 16). Unavailable dimensions are zero-filled with explicit availability flags, preserving a deterministic feature layout across training and inference.

Feature processing and dimensionality. BenchScout uses domain-aware scaling rather than global MinMax/z-score normalization: static capacities keep physical magnitudes, dynamic counters become ratios or per-second rates, categorical and failure indicators are one-hot encoded, and missing values are zero-filled for deterministic inference. When `perf`, eBPF, or GPU telemetry is unavailable, the collector records degradation flags and zero-masks affected dimensions instead of aborting. The context contains roughly 80–120 numeric capacity and pressure features. This context/probe separation mirrors the Mixture-of-Experts pattern, in which a gating network conditions on global state while each expert observes task-local signals.

3.3 Router: Conditional Expert Selection

Because sub-benchmark outputs are correlated projections of the same system state (Section 2.2), a small diverse subset can suffice for full-suite reconstruction. The selected subset is not fixed: the router's scores can vary with observed CPU, memory, and I/O context. BenchScout therefore treats benchmark acceleration as context-conditioned expert selection: given $\mathbf{x}$, the router selects sub-benchmarks according to its learned score.

Expert Routing. Each executable sub-benchmark is a measurement expert in a fixed pool $\mathcal{E} = \{e_1, e_2, \ldots, e_n\}$. Given $\mathbf{x}$, a scoring model f_θ assigns each expert a score $s_i = f_\theta(\mathbf{x}, e_i)$, which is normalized into a selection probability $p_i = \exp(s_i)/\sum_{j=1}^{n} \exp(s_j)$. The router then executes the Top-K experts $\mathcal{S}_K(\mathbf{x}) = \text{TopK}_{e_i \in \mathcal{E}}\, p_i$, and the resulting partial observation vector is passed to the reconstructor to estimate the full-suite profile.

Training Objective. For each run i, all experts share context $\mathbf{x}_i$ and receive label $y_{i,j}$ from $\mathbf{y}_i$. The LightGBM router minimizes a pairwise ranking loss using the within-run ordering of raw component scores; neural routers minimize cross-entropy to normalized component scores $\{y_{i,j}/\sum_k y_{i,k}\}$. These objectives rank higher-scoring components; they do not directly optimize reconstruction error or the information gained by probing an expert. Suite recovery is delegated to the reconstructor. Budget K trades accuracy for cost: smaller K saves time but leaves fewer observations for reconstruction.

Router architectures and inference. We implement three router families: a **LightGBM ranker** treating each $(\mathbf{x}, e_i)$ pair as a ranking instance, and **MLP** and **GNN-expert** routers that embed $\mathbf{x}$ and emit a softmax-normalized logit vector p_i. Their supplementary model-grid comparison is retained in the experiment artifacts. At inference the router selects the Top-K experts by descending p_i, conditioned solely on pre-run context $\mathbf{x}$—never on probe outcomes—so the order Profile → Route → Probe → Reconstruct is preserved. In the audited corpus, each applicable host–suite pair has one collection session with five runs; this does not provide 15–60 independent labeled sessions.

3.4 Time-Bounded Probing

Beyond horizontal subset selection, BenchScout reduces per-expert cost via time-bounded probing: a fixed wall-clock window replaces run-to-completion execution, and a supervised estimator maps prefix telemetry to the official full-run score (Section 2.2).

Dual-Mode Probing. **Micro-probes** map expert e_j to a coarse category (CPU, memory, I/O, syscall, thread, GPU) and run a short synthetic stimulus (e.g., sqrt loop for *dhry2reg*, temp-file I/O for *fstime*) while collecting eBPF/`/proc` telemetry. **Real-probes** invoke the original subtest under a hard timeout. Both emit bounded telemetry $\mathbf{p}_{i,j}$ and predicted score $\hat{y}_{i,j}$; supervision always targets the official full-run label $y_{i,j}$. Formally, for run i and expert e_j, a per-expert estimator predicts $\hat{y}_{i,j} = f_{e_j}(\mathbf{p}_{i,j}^{\text{micro}})$; in the shared-estimator setting, $\hat{y}_{i,j} = f(\mathbf{p}_{i,j}^{\text{micro}}, \text{id}(e_j))$, where $\text{id}(e_j)$ one-hot-encodes expert identity. Production telemetry is often incomplete; BenchScout encodes anomalies as features rather than discarding runs:

- **Truncation:** timeout events set *real_timed_out* and record elapsed wall time.
- **eBPF fallback:** when tracing is unavailable, *ebpf_available*=0 and eBPF dimensions are zero-masked.
- **Rate normalization:** monotonic counters (*sched_switch*, *syscall_enter*) convert to per-second rates.

Parallel probe collection. Each probe runs three concurrent activities within the fixed wall-clock window T_{probe}—an eBPF sampling thread (when permitted), the workload stimulus, and a trailing `/proc` snapshot—capturing kernel-level scheduling dynamics and user-space resource usage under the same budget rather than serializing telemetry after completion. Real-probes invoke the benchmark subtest under a hard `timeout`, recording *real_timed_out* and elapsed wall time as features instead of aborting; the implementation defaults to micro-probes for predictable latency and supports real-probes when fidelity to the original binary path is required.

Training objective. The estimator minimizes $\mathcal{L}_{\text{probe}} = \sum_{i,j}(\hat{y}_{i,j} - y_{i,j})^2$ in official label space (UnixBench indices; $\log(1+y)$-transformed PTS primaries), yielding fixed T_{probe} at inference. Probe estimators are trained per host alongside the router and reconstructor; during BenchScout inference, only the K router-selected experts invoke live probes, while the remaining $n-K$ component scores are imputed exclusively by the reconstructor from the sparse tuple Z.

3.5 Reconstructor: Full-Suite Result Recovery

Given system context and sparse probe-imputed partial scores, the reconstructor is a multi-output regressor that predicts all unexecuted component scores and the aggregate suite index. Its input deliberately interleaves global context $\mathbf{x}$ with per-expert tuples (m_i, v_i, τ_i) so that imputation can exploit both pre-run system state

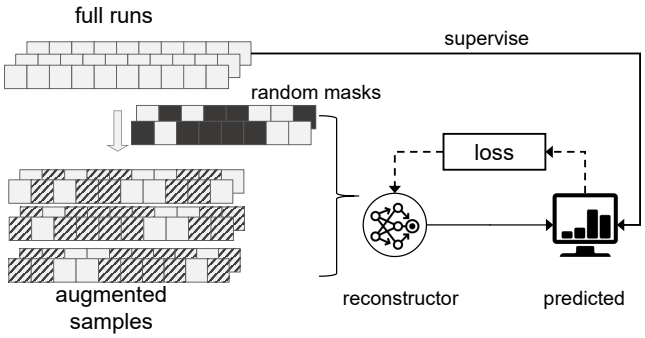


Figure 3: Reconstructor training with random-masked full runs. Historical full-run results are randomly masked to generate augmented sparse samples. The reconstructor takes the masked scores and binary mask as input, predicts the full-run result, and is optimized using the original full-run result as the supervision target.

and which experts were actually probed, for how long, and with what estimated scores.

Input Representation. Each expert e_i is a 3-tuple $z_i = (m_i, v_i, \tau_i)$ with mask m_i, score v_i, and probe wall time τ_i (zero-filled if unexecuted); the reconstructor input is the concatenation $X = [\mathbf{x}, z_1, \ldots, z_n]$. Target $Y = [r_1, \ldots, r_n, S]$ includes all component scores and suite index S; tree models use independent regressors per dimension.

Training and aggregation. Historical full runs are augmented with random masks (Figure 3) and supervised with $\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 + (\hat{Y} - Y)^2$. UnixBench labels use per-subtest *Index* scores and the published *System Benchmarks Index Score*. For PTS, the profiles expose primary throughput or latency measurements but do not provide a single aggregate score used by this paper; we define a paper-specific proxy $A_{\text{logmean}}(\mathbf{v}) = \exp(\frac{1}{N}\sum_{j=1}^{N}\log(1+v_j)) - 1$ over the included profile values. This proxy is not an official PTS suite score. Probe and reconstructor inputs for PTS use $\log(1+y)$ scaling; the proxy is used as the PTS suite-level training and evaluation target. The probe-only ablation, which has no reconstruction head, combines predicted UnixBench component indices using the smoothed geometric mean $\exp(\text{mean}_j \log(1+\hat{y}_j)) - 1$. This diagnostic aggregation differs slightly from UnixBench's published geometric mean of component indices; the full pipeline instead predicts the published suite index directly.

Mask augmentation and training input. The generic reconstruction path can augment historical full runs by randomly masking component scores (Figure 3): a random subset of size $k \sim \text{Uniform}\{1, \ldots, n\}$ is marked observed (m_i=1, v_i=y_i), with unobserved entries set to zero. This creates noiseless partial scores and does not itself model probe bias. For the headline UnixBench hybrid OOF evaluation, the reconstructor instead trains on inner-fold out-of-fold probe predictions and their probe durations; outer-fold evaluation supplies predictions from the probe model fit on the outer training runs. At inference, observed slots likewise carry probe-predicted scores $\hat{y}_j$ and measured durations τ_j. Tree-based reconstructors train one regressor per output dimension, handling heterogeneous score scales without shared output layers, while neural reconstructors use a shared trunk with multi-head outputs; both optimize the same $\mathcal{L}_{\text{recon}}$ against official full-run labels.

End-to-end information flow. Figure 2 summarizes how the three modules compose: the router maps $\mathbf{x}$ to a discrete expert subset, probe estimators map prefix telemetry $\mathbf{p}_j$ to scores for observed experts, and the reconstructor maps the concatenated tuple Z to the full vector $\hat{\mathbf{y}}$ and a separately predicted aggregate $\hat{Y}$. The headline hybrid OOF protocol trains reconstruction on out-of-fold probe predictions, but the available evaluation does not establish that reconstruction reliably corrects probe bias; the random-mask training path uses noiseless observed scores.

Web curve instantiation. The Web specialization uses the same router–probe–reconstructor roles: the router selects offered-load coordinates, the probe is the real Faban cell, and the reconstructor fills omitted curve entries. The UnixBench probe-score model is not reused because Web cells already expose target throughput directly. For Web profiles, the coordinates are five offered-load levels, with a budget of $K = 3$ target measurements capped at 60 s each. The nested selector evaluates all ten three-load subsets by inner session-held-out curve MAPE using training sessions only. The adaptive selector starts at the lowest offered load and greedily adds the load maximizing $|\hat{z}_j|\, s_{j|S}$, where $\hat{z}_j$ is the correlation model's predicted standardized deviation from the training mean and $s_{j|S}$ is the conditional standard deviation given the already selected loads S. The correlation estimator uses training-fold means, scales, and correlations; matched controls use linear interpolation with endpoint extrapolation or least-squares scaling of the training-fold mean curve. Sessions remain the split unit, and cold-start folds train on the other PHP-FPM setting. This is the Web workload adapter of the same budgeted-profile objective: its selector and estimator are specialized to ordered load curves, whereas the suite implementation selects benchmark components and predicts their scores.

4 Evaluation

4.1 Experimental Setup

Testbeds. The audited CPU corpus contains five Linux host identities (Table 1), referred to by the compact CPU×RAM shorthand used throughout this section. The GPU comparison also includes the supplied M6/NVIDIA T4 result; CPU and memory specifications for M6 were not reported. M1 (32 vCPU, Intel Core i9-14900K, 128 GiB, NVIDIA RTX 3090 Ti) and M6 (NVIDIA T4) have available GPU comparison values; M2–M5 are CPU-only in the audited corpus (Intel Xeon 6982P-C or Xeon Platinum).

Feature collection. System context $\mathbf{x}$ is collected with a 3 s warmup and 0.5 s `/proc` sampling; when perf PMU counters are blocked the collector falls back to `/proc`-based proxies without aborting, and GPU dimensions are zero-masked on CPU-only hosts—mirroring the degradation paths of the online pipeline (Section 3.2).

Benchmark suites. We use three suites that stress complementary subsystems. **UnixBench** (UB) is a classic system-level suite with N=12 single-copy subtests spanning CPU kernels, filesystem I/O,

Table 1: Testbeds represented in the evaluation. M1–M5 are the audited Linux CPU corpus; M6 is included for its supplied NVIDIA T4 GPU result. M6 CPU and RAM specifications were not reported.

Host	vCPU (CPU model)	RAM	GPU
M1	32(Intel Core i9-14900K)	128GB	NVIDIA RTX 3090 Ti GPU
M2	2(Intel Xeon 6982P-C)	8GB	—
M3	4(Intel Xeon 6982P-C)	8GB	—
M4	4(Intel Xeon 6982P-C)	16GB	—
M5	8(Intel Xeon Platinum)	8GB	—
M6	Not reported	Not reported	NVIDIA T4 GPU

and OS/interaction workloads; its aggregate is the *System Benchmarks Index Score*. **PTS-CPU** adds ≈13 compilation, cryptography, compression, and numerical profiles. We report their primary throughput or latency metrics and use the paper-defined log-mean proxy from Section 3.5 for suite-level comparisons. **PTS-GPU** adds OpenCL/GPU compute workloads. UnixBench and PTS-CPU run on M1–M5 in the audited raw corpus; available PTS-GPU comparison values cover M1 and M6. The audited raw CPU and M1 GPU corpus has one collection session of five runs for each applicable host–suite pair; the supplied M6 comparison has no accompanying run-count metadata here.

Web-serving workloads. We additionally evaluate two HTTP workloads, which are separate from the UnixBench/PTS host–suite corpus. The multi-tier CloudSuite Web Serving workload uses PHP, MySQL, and Memcached with Faban as the load generator on M1; its profile has five offered-load levels and two PHP-FPM settings (`max_children`=4 and 8). The strict curve analysis uses four valid held-out sessions per setting after excluding the session with a failed transaction, with session-level splits. A separate static-content Nginx workload runs on M1 over loopback, comparing `worker_processes`=1 and 4 at five offered loads in four randomized blocks (80 completed cells). The two HTTP workloads are reported separately because CloudSuite measures a multi-tier application while Nginx provides a lightweight static-content counterexample. Their collection and split protocols are detailed in Sections 4.4 and 4.5.

Baselines. The *full-run* baseline executes the entire suite and serves as the reference. The zero-probe train-fold mean is a same-host history baseline. Fixed-first, fixed-middle, and random subsets test how much a selected subset matters; router-only replays compare policies when selected component scores are known. We also evaluate two local execution-free proxies inspired by Wang et al. [41] and Tousi et al. [36]. These DNN and MLP implementations use the available static hardware features and are not reproductions of the cited methods, so they do not support conclusions about those method families. All predictive baselines use the same reported within-session folds. The execution-free proxies incur feature-collection cost (≈3.5 s), while BenchScout adds bounded probes. Since each host–suite has one five-run session, these comparisons do not test temporal or session-level generalization.

Evaluation protocol. We evaluate three execution paths—the BenchScout pipeline (main method: collect $\mathbf{x}$, router Top-K, probe the selected components, reconstruct the suite score), a *router-only* ablation that substitutes *measured* partial scores on the routed subset for probe predictions, and a *probe-only* ablation that probes every component without routing or reconstruction—plus supplementary studies on Top-K, features, telemetry, and probe duration. Every raw host–suite pair available to this study consists of one session with five runs. The reported cross-validation is a within-session random-fold analysis; folds contain very few test samples, and the repository audit found no machine-readable manifest for a locked train/validation/test split or session-level holdout. Thus model selection and generalization beyond these runs cannot be independently established from the available corpus. Models are fit separately per host, so these results do not measure cross-host transfer. Full-run labels and wall times come from previously collected runs, so the headline evaluation is an offline replay. Partial times in that analysis are budget estimates unless explicitly identified as online measurements.

Metrics. We report the suite relative error $|\hat{S} - S|/\max(|S|, \epsilon)$, where $\hat{S}$ is produced entirely by model inference and the reference index S serves only as the label, and the saved wall-time fraction $f_{\text{saved}} = (T_{\text{full}} - T_{\text{partial}})/T_{\text{full}}$. For Top-$K$ comparison we use a balanced score (lower is better) that prioritizes accuracy and treats time savings as a tie-breaker,

$$\text{Bal} = \frac{e}{e_{\min}} + \lambda\,(1 - f_{\text{saved}}), \tag{3}$$

where e is the mean out-of-fold suite relative error, $e_{\min} = \min e$ across the compared settings, and $\lambda = 0.15$. Feature-ablation importance is the relative-error increase $\Delta e_m = (e_m - e_{\text{full}}) \times 100$ percentage points when group m is removed.

4.2 System-Suite Profile Results

Table 3 reports five-fold run-held-out estimates from the historical corpus: five held-out runs per host–suite, or 25 predictions per suite across five hosts. Each fold fits the router, probe estimator, and reconstructor on the other four runs; reconstructor inputs are probe-estimator outputs $\hat{y}_j$, not measured scores from the held-out run. Fold-local LightGBM reconstructors use 30 estimators for PTS and 200 for UnixBench.

BenchScout achieves 0.21% median UnixBench suite-index error at an estimated 18.1 s context-plus-probe budget, comprising about 3 s of context collection and 15.1 s of selected probes. On the same 25 held-out runs, its 300 component predictions have 0.86% mean relative error (4.34% at the 95th percentile); restricting the denominator to the 225 unselected components gives 0.88% mean MAPE across runs. The corresponding omitted-component means are 2.29% for PTS-CPU (140 components over 25 runs) and 2.48% for the five-run M1 PTS-GPU audit. Full-suite times come from representative historical runs, so the table does not report paired end-to-end speedups. The static-feature proxies are local implementations, not reproductions of the cited methods.

The train-fold mean is a warm-start control that uses other runs from the same host–suite session and requires no new measurement. BenchScout has lower point error on 3/5 UnixBench hosts and 2/5 PTS-CPU hosts; history is at least as accurate on the others. These

Table 2: Live UnixBench routing-policy comparison on M1. One complete single-copy reference run (index 1816.3; 1,720.9 s) was followed by three randomized repetitions of fresh online micro-probes per policy. Each row averages three repetitions; learned selections vary by repetition. Probe wall is the sum of selected process wall times, while phase wall includes collection overhead. The host was concurrently loaded; raw probes, load averages, affinity, and reference report are in the artifact JSON.

Policy	Mean prediction	Mean err. (%)	Probe wall (s)	Probe phase (s)
Learned Top-3	3192.429	75.766	15.087	15.228
Fixed first-3	3192.429	75.766	15.066	15.290
Fixed CPU-mix	3192.429	75.766	15.070	15.439
Fixed I/O-mix	3192.429	75.766	15.100	15.230

are within-session estimates, and the reported time values combine nominal probe budgets with historical full-run times rather than paired end-to-end measurements.

For M1 UnixBench, additional selective-execution controls (Section 5.8, Table 15) show that subset selection and reconstruction can work well with measured component scores. These replay baselines and adaptive probing use different information or cost accounting, so they are not matched end-to-end competitors to probe-imputed BenchScout.

We also ran the four policies online on the current M1 host, rather than replaying archived probe records. One complete single-copy UnixBench run supplied a common live reference (index 1816.3; 1,720.9 s), followed by three randomized repetitions in which each policy collected three fresh 4-second micro-probes. Table 2 reports these measurements. The live reference ran under the concurrent host load recorded in the artifact, so this is a validation of the deployed pipeline under that operating condition, not a replacement for the within-session OOF estimate. The online predictions are much higher than the reference (75.77% mean error) and nearly identical across policies, exposing a temporal/load shift that the historical checkpoint does not absorb.

For comparison, the archived five-host OOF replay gives 0.590% mean error for all four policies and identical per-run suite predictions on the 25 held-out runs. This is a genuine null result, not duplicated measurements: the fold-local reconstructors map policy-specific partial inputs to the same suite estimate. A feature-use audit found zero probe-feature split gain in 22 of 25 suite-output heads, consistent with the reconstructor relying predominantly on host context for this target. The fold-matched context-only control has 0.608% mean error. Thus the historical folds show no accuracy advantage for learned routing, while the new online run reveals poor temporal/load robustness.

Interpretation. In the live run, all policies produce nearly the same prediction despite selecting different components; the current reconstructor is insensitive to the fresh probe values under this shifted operating state. The online result is therefore a measured robustness failure, not evidence that the policies are equivalent in general. The archived OOF audit likewise found zero probe-feature split gain in 22/25 suite-output heads and no accuracy advantage from learned routing. We retain both views because the historical folds and live deployment answer different questions.

PTS-GPU results. Table 4 reports a five-run, one-session M1 comparison on the paper-defined log-mean target; history and the static DNN proxy have lower error than the fold-based hybrid estimate (0.9534%). A separate online report gives 0.18% for M1 and 0.14% for M6/T4 at a reported 15 s. These values use a different protocol: M1's held-out target overlaps model selection and the 15.03 s probe time excludes 7.93 s context collection; M6 run-level logs are unavailable. We retain them as provenance-only case-study values in Appendix B, separate from the fold comparison.

The supplied online GPU comparison is kept distinct from the run-held-out table: M1's target run was included in the model-selection corpus, and the T4 run-level logs are unavailable. The 15-second figure is selected-probe time; M1 adds 7.93 seconds of context collection, for 22.965 seconds total hybrid wall time. These measurements are useful as concrete case studies on two GPU hosts, but they are not independent estimates of generalization.

4.3 Transfer and Temporal Diagnostics

The in-corpus folds estimate within-session interpolation, so we separately inspect one later M1 run and an offline host-held-out replay. These diagnostics ask whether the learned policy travels beyond the rows used to fit it; they are not pooled into the headline accuracy estimates.

On 2026-10-12, checkpoints saved on June 10 or earlier predicted 3189.654 for a newly collected M1 UnixBench run whose full-run target was 3418.5 (6.6943% error, $n = 1$). Router scores tied across all 12 subtests, triggering the implementation's directory-order fallback to *dhry2reg*, *whetstone-double*, and *execl*. Context collection took 8.699 s and the three real probes took 3.012 s. This temporal observation shows a material gap between excellent within-session folds and a later run; the tied router and single observation preclude a generalization claim. The records also show a scikit-learn 1.8.0-to-1.7.2 checkpoint compatibility warning.

In an offline leave-one-host-out replay, test hosts were excluded from router, probe-model, and reconstructor fitting; source-host reconstruction used inner host-held-out probe fits. Probes were archived from full-run collections rather than freshly executed on the held-out hosts. The four compared policies average 15.07% mean host error and 40.44% worst-host error, with mean errors within 0.002 percentage points of one another; predicted suite indices differ by at most 0.079 over 25 runs. Thus host transfer remains difficult, and current evidence does not show the learned policy improving over alternatives under host shift.

An oracle subset diagnostic isolates the value of choosing components when their full-run scores are already known. A training-host score-ranked subset has 4.49% mean error (0.98–10.25% across hosts), versus 9.53% for random, 12.57% for first-three, 16.26% for context-only Ridge, and 22.60% for training mean. This indicates that target-host component observations can be informative, but it is an upper-bound diagnostic: it does not validate pre-run routing, short probes, or online transfer.

Table 3: Overall effectiveness on CPU benchmark suites across five heterogeneous CPU hosts (M1–M5). BenchScout rows use the full pipeline: router Top-K, *probe-model-predicted* partial scores, and reconstruction (Section 4.2). Wang-style DNN and Tousi-style MLP rows are local execution-free proxies inspired by [36, 41], using static hardware features only; they are not reproductions of the cited methods. PTS-CPU proxy errors use the same log-mean targets and run-held-out folds as BenchScout. Time is an estimated new-measurement budget in seconds: zero for history, about 3.5 s for static-feature collection, and 18.0–18.1 s for BenchScout context plus selected probes. Full-suite entries are historical wall times, not paired timing controls. Bold marks the lowest error in each host column.

Suite	Method	M1 (32U128G)		M2 (2U8G)		M3 (4U8G)		M4 (4U16G)		M5 (8U8G)	
		Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)
UnixBench	Full-suite	–	1691.0	–	1520.0	–	1580.0	–	1610.0	–	1550.0
	Train-fold mean	0.21	0.0	**0.19**	0.0	0.35	0.0	**0.13**	0.0	2.78	0.0
	Wang-style DNN proxy	0.21	3.5	2.67	3.5	6.28	3.5	4.47	3.5	4.37	3.5
	Tousi-style MLP proxy	1.7	3.5	99.56	3.5	99.63	3.5	99.60	3.5	99.59	3.5
	BenchScout	**0.18**	18.1	0.29	18.1	**0.34**	18.1	0.19	18.1	**1.95**	18.1
PTS-CPU	Full-suite	–	15755.0	–	12450.0	–	13100.0	–	13900.0	–	12800.0
	Train-fold mean	1.91	0.0	**0.16**	0.0	**0.18**	0.0	**0.21**	0.0	0.22	0.0
	Wang-style DNN proxy	1.91	3.5	0.26	3.5	68.53	3.5	0.22	3.5	**0.17**	3.5
	Tousi-style MLP proxy	1.88	3.5	99.42	3.5	99.93	3.5	97.57	3.5	98.77	3.5
	BenchScout	**1.74**	18.0	0.19	18.1	0.26	18.0	0.30	18.0	0.19	18.0

Table 4: M1 PTS-GPU run-held-out comparison on the paper's log-mean target (five runs, one session). All rows use the same held-out runs. History mean uses the other four training labels; execution-free proxies use static hardware features; BenchScout uses host context and selected probes. Cost is the estimated new measurement budget, not model-training or end-to-end serving time.

Method	Mean err. (%)	Cost (s)
Train-fold history mean	0.7993	0
Wang-style DNN proxy	**0.7987**	3.5
Wang-style linear proxy	0.8492	3.5
Tousi-style random forest	0.8695	3.5
Tousi-style decision tree	0.8492	3.5
Tousi-style MLP	0.8592	3.5
Tousi-style elastic net	0.8298	3.5
BenchScout full pipeline	0.9534	18.0

Table 5: Supplied online PTS-GPU case studies. Values are separate from run-held-out results: the M1 target overlaps model selection and M6/T4 logs are unavailable. Cost denotes selected-probe time; M1 context collection adds 7.93 s.

Suite	Method	M1 (RTX 3090 Ti)		M6 (T4)	
		Err. (%)	Cost (s)	Err. (%)	Cost (s)
PTS-GPU	Full-suite	–	27018.0	–	12951.0
	Wang-style DNN proxy	0.26	3.5	5.13	3.5
	Tousi-style MLP proxy	98.85	3.5	67.14	3.5
	BenchScout	**0.18**	15.0	**0.14**	15.0

4.4 Multi-tier CloudSuite Web Profile

CloudSuite Web Serving is a multi-tier application with PHP, MySQL, and Memcached; all tiers and the Faban load generator run on M1 over a private Docker bridge. We compare PHP-FPM `max_children=4` and 8 over five offered-load levels, with four independently restarted sessions per configuration. Each session contains 60-second probe cells and 120-second full-reference cells with fresh services between cells. We hold out a complete session and fit selectors and curve estimators on the other three; the session, not an individual load cell, is the evaluation unit. A session enters the strict primary analysis only if all 20 configuration, phase, and load cells are present, have valid operation mixes, and have no failed transactions. Sessions 1, 2, 4, and 5 pass this rule. Session 3 has one failed Register transaction and is excluded from the primary analysis; it is retained only in a separately labeled one-failure sensitivity analysis. The locked split is recorded in the session-audit JSON under the round-8 results directory (input hash prefix `ff3bc3c19b75`; the full hash is in the artifact). This CloudSuite instantiation uses workload-specific load selection and sparse-curve reconstruction to measure the accuracy–cost trade-off directly; its selector and estimator are analyzed as a distinct Web workload component of the measurement-compression study.

For this CloudSuite/Faban workload, operation-level latency limits are p95 $\leq$ 500 ms and p99 $\leq$ 1,000 ms. We report these as a workload-specific feasibility screen separate from the primary throughput-profile fidelity endpoint; they are not a general prerequisite for the system-suite experiments.

Every probe policy receives the same three 60-second load measurements per configuration. The primary same-configuration analysis trains and tests on the same PHP-FPM setting using different sessions. A separate cold-start analysis transfers estimates from the other setting and measures curve error only; it is not used to claim a configuration-choice result. We report the two transfer directions separately because their profile shapes differ. Controls include training-session history, nested and adaptive selection, fixed first/middle/evenly spaced loads, interpolation, history scaling, and the exact expectation over all ten random three-load subsets. Table 6 reports reconstructed policy costs from the complete collection grid; the separate paired FPM4 selective-phase timing is reported below.

The same-configuration history-only baseline has 5.26% full-curve MAPE and 5.26% MAPE on the ten unselected load cells; adaptive correlation has 6.40% full-curve and 5.31% unselected-load MAPE (four held-out sessions). Session-resampling bootstrap 95% intervals for the unselected metric are [3.80,7.66]% for history and [2.03,10.28]% for adaptive correlation. Thus probing does not

Table 6: CloudSuite Web Serving profile comparison over four strict held-out sessions (1, 2, 4, and 5). Predictors use either zero probes (history mean) or the same $K = 3$ budget of 60-second cells. Same-configuration and cold-start report five-load curve MAPE; cold-start averages both transfer directions. Costs sum measured selected-cell runner and total times from the complete collection grid.

Selector	Curve estimator	K	Same-config MAPE (%)	Cold-start MAPE (%)	Runner / cell total (s)
History mean	Training-session mean	0	5.26	39.16	0 / 0
Nested train-only	Correlation	3	6.04	19.61	453 / 476
Adaptive correlation	Correlation	3	6.40	17.35	442 / 466
Adaptive correlation	Linear interpolation	3	7.61	7.21	442 / 466
Fixed middle loads	History-scaled curve	3	5.63	13.98	454 / 478
Fixed evenly spaced loads	Linear interpolation	3	7.91	7.91	455 / 478
Fixed evenly spaced loads	History-scaled curve	3	5.93	14.25	455 / 478
Uniform random, exact expectation	History-scaled curve	3	5.92	14.53	455 / 478

improve stable-configuration recovery, and fixed history-scaled subsets are competitive. The complete-grid comparison reconstructs three-cell cost as 442 s runner/466 s total versus 1,100 s for full-reference cells. Separately, a pre-fixed evenly spaced linear interpolation policy was run in four valid exploratory FPM4 sessions, using probe loads 20/60/100 and same-session full references at 20/40/60/80/100. Sessions 6 and 8 use 60-second probes followed by 120-second references; sessions 9 and 14 use 60-second full references first, then 60-second probes. Omitted-load MAPE is 5.18%, 9.77%, 19.93%, and 13.64% (mean 12.13%); full-curve MAPE is 5.28%, 10.01%, 14.61%, and 19.45% (mean 12.34%). Probe/full cell totals are 481.710/1,093.760, 475.056/1,086.334, 472.638/792.409, and 545.056/827.209 s, respectively. All 32 cells are valid with zero operation failures. Table 7 gives session-level results. Sessions 9 and 14 are matched-duration, full-first stress cases; reverse ordering and temporal drift remain confounds. With $n = 4$, these are exploratory joint accuracy/cost observations, not evidence that isolates probe-count effects or establishes general speedup.

The complete-grid comparison reconstructs three-cell cost as 442 s runner/466 s total versus 1,100 s for full-reference cells. A separate Session 5 phase audit paired five probe cells with five full-reference cells at identical loads: FPM4 totals were 790.86/1,119.02 s and FPM8 totals were 795.10/1,094.00 s (probe/full). These phases include setup/reset per cell and compare all five loads, so they provide paired phase accounting rather than $K = 3$ end-to-end savings.

Sessions 11 and 12 were launched concurrently on the same host. Session 11 had no transaction failures but four operation mixes failed the strict tolerance; session 12 passed the mix rule. Their 31.58–41.89% omitted-load errors are reported as contention sensitivity, not pooled into the isolated mean. Session 13 used a different subset (20/40/100), with full-first 60-second cells; seven of eight cells passed, but full load 100 failed the mix rule. The two valid omitted-load references (60/80) yielded 20.63% MAPE under linear interpolation from probes 40/100. Its incomplete profile is excluded from the paired aggregate.

For cold-start transfer, history-only error is 39.16% full-curve and 39.16% on unselected loads; adaptive correlation with linear interpolation reaches 7.21% full-curve and 8.74% unselected-load MAPE, while fixed evenly spaced interpolation reaches 7.91% and 8.94%, respectively (four held-out sessions). The session-level ordering varies, so these data show improved recovery after a configuration shift but do not establish adaptive superiority. All 80 strict cells have valid mixes and zero transaction failures; the workload-specific latency screen is reported separately and is not a general system endpoint.

Across the four held-out sessions, descriptive percentile-bootstrap 95% intervals for unselected-load MAPE are [7.39,10.08]% for adaptive interpolation and [7.53,10.35]% for fixed-even interpolation; with only four independent sessions these intervals are uncertainty summaries, not superiority tests. The complete session-5 run pairs each probe cell with a full cell at the same load (Appendix B); sessions 6, 8, 9, and 14 report valid exploratory accuracy/cost pairs above, while session 13 is retained as an incomplete strict-cell sensitivity.

In post hoc cold-start controls, each of five fixed single-load choices with history scaling makes zero wrong choices in the same eight throughput-only switch decisions, but curve MAPE ranges from 15.96% to 33.76% (102–201 s mean runner time), compared with 7.21% for three-load interpolation. The observed benefit is profile fidelity rather than a demonstrated tuning gain.

4.5 Static-content Nginx Comparison

We compare two M1 Nginx configurations (`worker_processes=1` or 4) at five offered loads over four randomized blocks (80 completed cells). The loopback server uses port 18081; manifests and raw logs are in the artifact.

For each held-out block, selection uses the other three blocks. The probe policies receive three 4 s load measurements per configuration; controls include fixed and random subsets and the zero-probe history mean (Table 8). Across eight held-out profiles, nested selection has 2.12% throughput-curve MAPE, while history-only has 1.40%.

Nested probing and history-only both select `workers4` in all four blocks. Table 8 shows that history is also better on aggregate throughput MAPE, while Table 9 shows the same ordering for the descriptive p99 profile. These counterexamples sharpen the scope: sparse probing is not uniformly better when the host/configuration history already predicts a stable profile.

5 Ablation Study

This section examines how measurement budget, model choice, context, probe quality, telemetry, and probe duration affect the system. Unless stated otherwise, M1 replay diagnostics use one canonical five-run session and directly observed selected-component scores to isolate router and reconstruction behavior from probe-estimation

Table 7: Paired FPM4 accuracy and phase cost for a pre-fixed evenly spaced interpolation policy. Each prediction uses probe loads 20/60/100 and is scored against same-session full references at 20/40/60/80/100. Cell totals include per-cell setup/reset and Faban wall time.

Session	Omitted MAPE (%)	Full-curve MAPE (%)	Probe phase (s)	Full-reference phase (s)	Saved (%)
6	5.18	5.28	481.710	1,093.760	56.0
8	9.77	10.01	475.056	1,086.334	56.3
9 (60/60, full-first)	19.93	14.61	472.638	792.409	40.4
14 (60/60, full-first)	13.64	19.45	545.056	827.209	34.1
Mean (valid, $n = 4$)	12.13	12.34	—	—	—
11 (concurrent stress; mix-invalid)	41.89	28.00	567.057	854.288	33.6
12 (concurrent stress)	31.58	33.43	580.475	865.119	32.9

Table 8: Held-out HTTP throughput-profile baselines across eight configuration-block profiles (four blocks, two configurations). Each probing policy uses three 4-second observations; cost is nominal `wrk` time.

Policy	Throughput MAPE (%)	Probe (s)
History mean	1.395	0
Fixed first three	2.530	12
Fixed middle three	2.792	12
Uniform random $K = 3$	2.910	12
Nested train-only $K = 3$	2.119	12

Table 9: Descriptive held-out p99 curve MAPE for the static-content Nginx counterexample. The profile has few successful operations per load and is not a production tail-latency guarantee.

Policy	p99 MAPE (%)
History mean	14.024
Fixed first three	27.809
Fixed middle three	30.113
Uniform random $K = 3$	40.186
Nested train-only $K = 3$	29.625

error; they complement, rather than replace, the probe-imputed headline evaluation.

5.1 Runtime Overhead Analysis

Table 3 places historical full-suite wall times beside nominal OOF probe budgets; these are not paired wall-time savings measurements. It does not show how CPU and memory evolve during execution. We therefore capture online resource traces (0.5 s sampling, t=0 at run start) on three representative hosts—M1 (32U128G), M3 (4U8G), and M4 (4U16G)—under four UnixBench conditions: *Full run* (complete *perl Run -c 1*), *router only* (Top-K partial execution), *probe only* (twelve D=4 s micro-probes), and *BenchScout* (router, Top-K probes, and reconstruction in the traced window; **x** collection precedes the trace). Figure 4 overlays the corresponding CPU and memory waveforms on a shared ≈1680 s axis. In these traces, the BenchScout activity appears in the early portion of the window, while the full UnixBench run continues across the full horizon; this descriptive comparison does not quantify application-level interference or establish negligible overhead. The underlying single-run timing records show full-suite wall times of 1701.7 s (M1), 1681.0 s (M3), and 1678.9 s (M4), versus 12.05 s for the BenchScout trace on each host. System-context collection occurred outside these traces, so 12.05 s excludes that stage; these runtime traces are separate from the held-out accuracy evaluation and are not paired with its predictions.

5.2 Model Combination Study

The *router only* path isolates the router–reconstructor stack by replacing probe-predicted scores with measured scores. This removes probe estimation error and tests whether a router-selected subset of true component scores is sufficient to reconstruct the full suite profile. We exhaustively evaluate all combinations of three routers (LightGBM, MLP, GNN-expert) and three reconstructors (XGBoost, LightGBM, MLP)—nine combinations per suite on M1—under online partial execution plus reconstruction. Table 10 reports suite relative error (%) and benchmark wall-time saved (%) for UnixBench, PTS-CPU, and PTS-GPU.

The grid reveals that reconstruction is the dominant source of reliability. Across routers, tree-based reconstructors (XGBoost and LightGBM) are consistently stable, while MLP reconstruction fails catastrophically in several cases, with relative errors far above 10^2%. This is because the reconstructor operates on sparse tabular inputs with structured missingness from Top-K routing, limited host-local training data, heterogeneous score scales, and nonuniform component correlations. Boosted trees provide a better inductive bias for this regime: they capture thresholded feature interactions and remain stable under sparse, small-data patterns. In contrast, MLP reconstructors must learn these conditional imputations from limited masked examples and can extrapolate poorly when the observed subset changes.

We therefore select the best *router only* combinations with tree-based reconstructors on M1: MLP router with LightGBM reconstructor for UnixBench (0.13% error) and PTS-CPU (0.22%), and LightGBM router with LightGBM reconstructor for PTS-GPU (1.60%). The *router only* supplementary studies in Sections 5.4 and 5.5 reuse these best M1 pairs, while the *probe only* evaluation in Section 5.3 uses independently trained probe regressors.

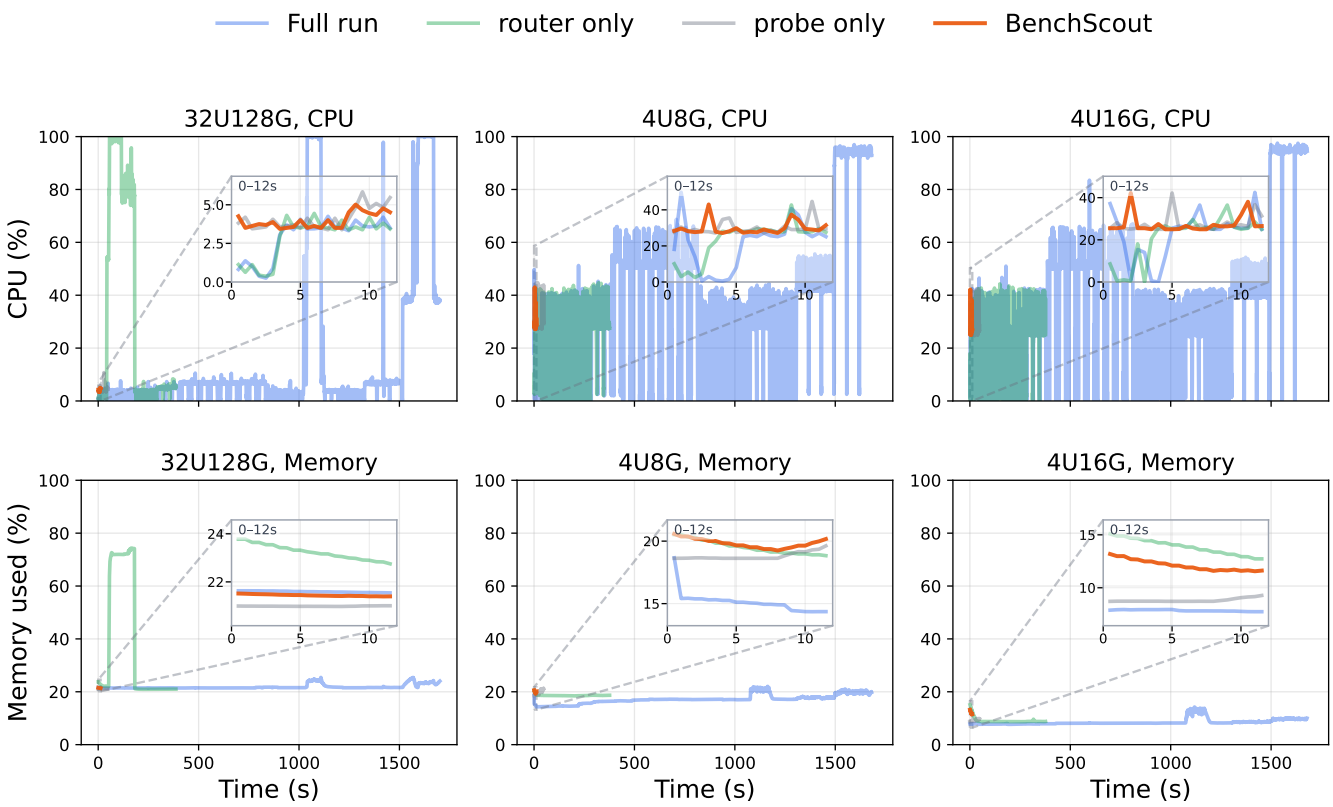


Figure 4: UnixBench CPU and memory waveforms on M1, M3, and M4 (0.5 s sampling). Rows: CPU utilization (%) and memory used (%); columns: 32U128G, 4U8G, and 4U16G. Curves show *Full run*, *router only*, *probe only*, and *BenchScout* on shared axes (≈1680 s horizon). Dashed guides and per-panel insets magnify 0–12 s, isolating the early window in which *BenchScout* completes while the full suite remains active for the entire trace.

5.3 Probe Model Evaluation

This experiment evaluates whether short, time-bounded probes can provide sufficient signal for suite-level performance prediction. Each component is executed only in micro-probe mode for a fixed budget of 4 s, optionally collecting eBPF features, and the resulting probe features $\mathbf{p}_{i,j}$ are used to predict component scores. Unlike the BenchScout execution pipeline, this experiment does not perform router-based subset selection or reconstruction; instead, it probes every component and directly aggregates the predicted component scores into a suite score.

We train two probe regressors, LightGBM and XGBoost, using probe features collected from each host's historical full runs, and evaluate online prediction against the latest full run on the same host. For PTS suites, labels use $\log(1+x)$ scaling with per-profile estimators to handle heterogeneous metric magnitudes. The evaluation covers UnixBench and PTS-CPU on all five audited hosts, plus supplied PTS-GPU values for M1 and M6. The repository OOF artifacts cover 11 audited host–suite pairs; the supplied M6 GPU comparison is reported separately.

Table 11 compares LightGBM and XGBoost on M1 across all three suites. It reports suite relative error, probe wall time T_{partial}, full-suite wall time T_{full} from the ground-truth run JSON, and the resulting wall-time savings. On M1, XGBoost consistently achieves lower suite relative error than LightGBM while providing the same wall-time savings, indicating that micro-probes provide useful predictive signal even without selective execution or reconstruction.

These supplementary online probe controls use a separate M1 canonical full run ($T_{\text{full}} = 319.9$ s) from the five-run OOF timing table; the two timing contexts are not directly interchangeable.

5.4 Top-K Sweep

Using the best *router only* router–reconstructor pairs on M1 from Section 5.2 (MLP + LightGBM for UnixBench and PTS-CPU; LightGBM + LightGBM for PTS-GPU), we sweep K from 1 to 6, capped by each suite's component count. For each K, the router selects Top-K experts, partial results are observed, and the reconstructor predicts the suite score. Evaluation uses within-session random-fold CV on the canonical M1 session (the available fallback because only one session is present), with router checkpoints and reconstructors from the associated *trained_models* directory; reconstructors train on partial subsets of size K. These five-run folds are exploratory and do not provide session-level validation; the reported supplementary analyses should not be read as independent confirmation of the headline model selection. Figure 5 visualizes the resulting error–

Table 10: Router–reconstructor combination study on M1 with online partial execution and reconstruction.

	Reconstructor					
	XGB		LGBM		MLP	
Router	Err.	Save	Err.	Save	Err.	Save
UnixBench						
LightGBM	0.93	72.5	0.62	76.3	43.8	76.5
MLP	0.34	78.2	**0.13**	72.1	42.8	79.1
GNN-expert	0.28	76.5	0.42	76.5	59.0	76.5
PTS-CPU						
LightGBM	0.35	74.7	7.15	60.6	1.54×10^4	80.2
MLP	1.30	87.5	**0.22**	89.6	2.78×10^3	92.3
GNN-expert	3.12	95.5	3.14	39.6	1.65×10^4	79.5
PTS-GPU						
LightGBM	2.03	78.0	**1.60**	78.5	3.49×10^2	78.5
MLP	1.61	95.3	1.61	95.3	2.83×10^2	95.3
GNN-expert	1.88	95.4	**1.60**	94.9	4.03×10^2	95.3

Table 11: Probe model comparison on M1.

Suite	Model	Rel. err. (%)	T_{partial} (s)	T_{full} (s)	Saved (%)
UnixBench	LightGBM	0.66	48.0	319.9	85.0
	XGBoost	0.45	48.0	319.9	85.0
PTS-CPU	LightGBM	2.40	52.0	13018	99.6
	XGBoost	1.03	52.0	13018	99.6
PTS-GPU	LightGBM	2.60	40.0	27018	99.9
	XGBoost	2.28	40.0	27018	99.9

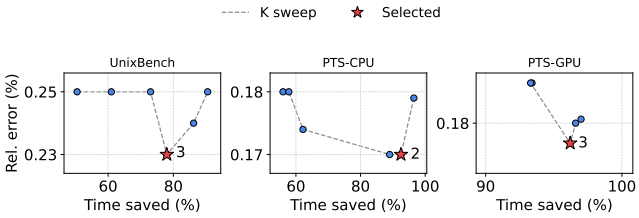


Figure 5: Top-K error–saving trade-off using the best router–reconstructor pair for each suite. Each point corresponds to a candidate K, with numbers indicating K. Stars mark the selected K according to the balanced score in Equation 3.

saving trade-off, where each point corresponds to one candidate K. Although the figure does not plot the balanced score directly, the selected K minimizes Equation 3, balancing suite relative error against wall-time savings. On M1, the selected points are at small K: K=3 for UnixBench (1.03), K=2 for PTS-CPU (1.01), and K=3 for PTS-GPU (1.01). These exploratory within-session results show the selected operating points under this score, but do not establish that the router finds generally informative probes or that the chosen K transfers to new sessions. Increasing K beyond these points provides limited error reduction in the analyzed folds while reducing estimated time savings.

Table 12: System-feature ablation at fixed router policy and $K = 3$ (offline CV on M1). Δe = relative-error change vs. complete x baseline (pp = percentage points; baseline row omitted). Importance rank orders ablated modes by descending Δe.

Suite	Ablation mode	Rel. err. (%)	Δe (pp)	Imp. rank
UnixBench	*static_hw_only*	0.241	−0.003	2
	no_perf_pmu	0.243	0.000	1
	no_dynamic_proc	0.241	−0.003	3
	no_gpu	0.237	−0.007	4
PTS-CPU	*static_hw_only*	2.04	+0.28	1
	no_perf_pmu	1.76	0.00	3
	no_dynamic_proc	2.04	+0.28	2
	no_gpu	1.75	−0.01	4
PTS-GPU	*static_hw_only*	0.957	+0.04	1
	no_perf_pmu	0.918	0.00	3
	no_dynamic_proc	0.957	+0.04	2
	no_gpu	0.818	−0.10	4

5.5 System Context Feature Ablation

To quantify which parts of the system context $\mathbf{x}$ drive reconstruction accuracy, we ablate feature groups while holding router Top-K selection at K=3 on M1 (leave-one-run-out CV on each suite's canonical session, random-fold fallback), using the same LightGBM reconstructor and *trained_models* checkpoints as in Section 5.4. We compare four ablated modes against a complete-$\mathbf{x}$ baseline (not shown): *static_hw_only* (hardware topology, memory, cache, and GPU inventory; zero dynamic/PMU), *no_perf_pmu* (remove PMU-derived rates except *perf_event_paranoid*), *no_dynamic_proc* (remove load, fault, and bandwidth-proxy dynamics), and *no_gpu* (remove GPU/OpenCL dimensions). Table 12 reports mean out-of-fold suite relative error and Δe_m relative to the complete-$\mathbf{x}$ baseline. Importance rank (1 = most important to keep among the four ablated groups) sorts ablated modes by descending Δe_m. On PTS-CPU, ablating static hardware or dynamic `/proc` proxies increases error by ≈0.28 pp; PMU and GPU groups have smaller effects. On UnixBench all Δe_m magnitudes stay below 0.01 pp on M1. On PTS-GPU, removing GPU/OpenCL inventory features lowers error by 0.10 pp. In this small replay, those features add noise rather than useful signal; the result cautions against treating hardware descriptors as uniformly beneficial inputs.

5.6 Robustness to Probe-Telemetry Restrictions

Section 3.4 and Appendix Table 16 include optional eBPF-derived scheduling and syscall rates in the in-run probe vector $\mathbf{p}_{i,j}$. These signals can improve observability on permissive systems, but they should not become a deployment prerequisite. In restricted environments where bpftrace is unavailable or tracing privileges are denied, BenchScout treats eBPF features as optional telemetry: it sets the

Table 13: eBPF degradation on M1 UnixBench *Probe only* (LightGBM; D=4 s; Y=3185.5). sched: mean context-switch rate (10^4/s).

Telem.	Err (%)	$\lvert\hat{Y}-Y\rvert$	$\hat{Y}$	T_{pr} (s)	eBPF ok	sched
On	0.27	8.48	3194.0	61.8	12/12	8.3
Off	0.17	5.42	3190.9	60.6	0/12	—

ebpf_available flag to zero and zero-masks the corresponding probe dimensions instead of aborting the probe or changing the feature layout. This design preserves a deterministic input schema while allowing the model to fall back to the remaining context and probe signals.

To evaluate robustness under realistic telemetry restrictions, we conduct a controlled *Probe only* experiment on M1 (32U128G) with UnixBench in which eBPF tracing is unavailable after training. We fix the trained M1 LightGBM probe regressor under the micro-probe setting (4 s per component) and run two back-to-back online conditions on the same host: **(i) eBPF on**, where bpftrace is enabled with elevated privileges; and **(ii) eBPF off**, where tracing is disabled and all eBPF dimensions are zero-masked. Each condition probes all 12 UnixBench components, predicts subtest indices, aggregates the suite score, and compares against the latest full-run ground truth in the canonical session (Y = 3185.5). The probe regressor is trained on M1 historical runs using the same micro-probe protocol and eBPF-enabled feature collection.

Table 13 shows that both conditions completed with sub-percent error, but zero-masking eBPF produced the lower error in this single run (0.17% versus 0.27% with eBPF available); probe times were comparable (60.6 s versus 61.8 s). Thus, this observation supports graceful operation when tracing is unavailable, not an accuracy benefit from eBPF. It does not establish end-to-end robustness across hosts, suites, contention levels, or other telemetry failures.

5.7 Probe Duration Sensitivity

Section 3.4 fixes a per-component probe budget as the primary knob for *Probe only* latency versus fidelity. We quantify this accuracy–cost frontier on M1 (32U128G) with a controlled UnixBench *Probe only* sweep over $D \in \{2, 4, 8\}$ s: for each duration, we collect micro-probes for all 12 components and train a dedicated LightGBM regressor (eBPF off to isolate duration effects) before online evaluation against the latest full run (Y=3185.5, T_{full}=319.9 s). Table 14 reports suite relative error and wall-time savings for each D; all 12 probes complete without timeout or pipeline failure. Error decreases monotonically with D while probe cost scales roughly linearly in D×12. At D=4 s, suite error is 0.56% with ≈81% wall-time saved, balancing accuracy against the 0.44% error at D=8 s and the lower probe cost at D=2 s (0.80%, 88.6% saved). This confirms the 4 s default used in Section 5.3 and the eBPF degradation study (Section 5.6).

5.8 Selective-Execution Baselines

The following M1 replay compares additional same-K policies and reconstruction methods omitted from the cross-host headline table. The rows do not all receive identical information: BenchScout uses its probe-imputed selected scores, while correlation and matrix completion use their respective replay inputs, and the adaptive-probing result comes from a separately configured within-session experiment. We report them to expose the quality range of simple and structured alternatives, not as a single controlled ranking.

Table 14: Probe duration sweep on M1 UnixBench *Probe only* (LightGBM; micro; eBPF off; Y=3185.5, T_{full}=319.9 s).

D (s)	Err(%)	$\lvert\hat{Y}-Y\rvert$	$\hat{Y}$	T_{pr} (s)	Saved(%)	Probe-ok
2	0.80	25.4	3210.9	36.4	88.6	12/12
4	0.56	17.8	3203.3	60.4	81.1	12/12
8	0.44	13.9	3199.4	108.6	66.1	12/12

Table 15: Additional M1 UnixBench selective-execution baselines (K = 3, within-session five-fold replay). Inputs and timing protocols differ by method; this table supplements, rather than replaces, the matched router controls in Table 2.

Method	Mean err. (%)	P95 err. (%)
BenchScout full pipeline	0.180	–
Fixed first-K subset	0.244	–
Random-K subset	0.236	–
Correlation reconstruction	0.770	0.928
Matrix completion	0.503	0.795
Adaptive probing	0.072	0.259

The 0.072% adaptive-probing row is numerically stronger than the 0.180% headline M1 value, which prevents a broad claim that the full pipeline is best on this host. It is a distinct replay protocol and should motivate a controlled follow-up rather than be hidden or treated as directly interchangeable.

6 Related Work

6.1 Performance Evaluation

Operating-system suites such as LMbench, UnixBench, and HBench-OS support robustness and embedded-system studies [6, 14, 16, 21, 22, 33, 34, 42]. Cost reduction includes suite selection, execution-time prediction, and adaptive stopping [18, 27, 31]. BenchScout estimates omitted entries of a specified profile from bounded target-host probes, comparing full-run, history, and fixed-subset baselines; its router ranks components by predicted score. Application suites and Web studies cover scale-out behavior, tuning, sampled-run models, and cloud search [2, 9, 12, 13, 38, 39]. Our Web endpoint reconstructs a five-load curve from three target measurements; fixed single-load controls match all throughput-only winners, so no tuning-benefit claim is made. The UnixBench aggregate is published and the PTS aggregate is a paper-defined log-mean proxy (Section 3.5).

6.2 Performance Prediction

Prior work lowers evaluation cost by predicting performance instead of measuring it. Simulation- and regression-based methods estimate unexplored design points from a fraction of simulated configurations [19, 20, 24, 32]; analytical models derive performance from explicit processor mechanisms such as pipeline stalls,

cache misses, prefetching, and MSHR limits [8, 10, 11, 15, 37]; and machine-learning models map program or hardware features to runtime or power on another platform (e.g., XAPP, LACross) or to new CPUs from public specifications [3, 36, 41, 43]. These approaches estimate unmeasured points from simulation or static features. BenchScout instead uses bounded target-host probes; its aggregate is the published UnixBench index or the paper-defined PTS proxy.

7 Limitations and Reproducibility

The system-suite evaluation uses host-specific models and run-held-out folds within one five-run session per host–suite; it does not establish future-day or unseen-host generalization. The Web study uses one host, four strict CloudSuite sessions per PHP-FPM setting (one failed session is sensitivity-only), and four randomized Nginx blocks, so it does not establish production validity.

The Web endpoint is profile fidelity: fixed single-load controls match all eight throughput-only winners, so no tuning-decision advantage is claimed. CloudSuite applies the same compression endpoint to offered-load coordinates; the component-suite path uses benchmark-component routing and probe-score prediction. Workload-specific latency screens are reported separately from profile accuracy.

System-suite probe budgets use historical full-run durations rather than paired end-to-end timings. The four isolated FPM4 pairs mix 60/120-second and matched 60/60-second full-first cells; ordering, drift, and $n = 4$ still prevent probe-count causal claims. Two concurrent full-first stress sessions had 31.58–41.89% omitted-load error and are reported separately; session 10 is excluded for invalid operation mix. CloudSuite costs are reconstructed, partial FPM8 sessions are excluded, PTS uses a paper-defined proxy, and the M6/T4 GPU case lacks run-level logs (Appendix B).

8 Conclusion

We study compressing cloud-infrastructure evaluation with bounded target measurements. UnixBench reaches 0.21% median error at an estimated 18.1 s context-plus-probe budget, while omitted-component MAPE is 0.88% for UnixBench and 2.29% for PTS-CPU. In CloudSuite's coordinate-specific cold-start profile recovery, three measurements reduce unselected-load MAPE from 39.16% for history-only to 8.74% with interpolation. Four isolated FPM4 pairs ($n = 4$) show 5.18–19.93% omitted-load MAPE and 5.28–19.45% full-curve MAPE; probe phase totals range from 472.638 to 545.056 s and full-reference phase totals from 792.409 to 1,093.760 s. Concurrent stress sessions expose higher error and are reported separately. The evidence supports measurement compression under the audited operating conditions: accuracy is retained for the reported folds and targets while the new measurement phase is bounded, with history and fixed policies exposed as credible alternatives.

References

[1] Alhamazani, K., Ranjan, R., Jayaraman, P. P., Mitra, K., Liu, C., Rabhi, F., Georgakopoulos, D., and Wang, L. Cross-layer multi-cloud real-time application qos monitoring and benchmarking as-a-service framework. *IEEE Transactions on Cloud Computing 7*, 1 (2015), 48–61.

[2] Alipourfard, O., Liu, H. H., Chen, J., Venkataraman, S., Yu, M., and Zhang, M. Cherrypick: Adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation* (2017).

[3] Ardalani, N., Lestourgeon, C., Sankaralingam, K., and Zhu, X. Cross-architecture performance prediction (xapp) using cpu code to predict gpu performance. In *Proceedings of the 48th International Symposium on Microarchitecture* (2015), pp. 725–737.

[4] Aslanpour, M. S., Gill, S. S., and Toosi, A. N. Performance evaluation metrics for cloud, fog and edge computing: A review, taxonomy, benchmarks and standards for future research. *Internet of Things 12* (2020), 100273.

[5] Baruwal Chhetri, M., Chichin, S., Vo, Q. B., and Kowalczyk, R. Smart cloudbench—a framework for evaluating cloud infrastructure performance. *Information Systems Frontiers 18*, 3 (2016), 413–428.

[6] Brown, A. B., and Seltzer, M. I. Operating system benchmarking in the wake of lmbench: A case study of the performance of netbsd on the intel x86 architecture. In *Proceedings of the 1997 ACM SIGMETRICS international conference on Measurement and modeling of computer systems* (1997), pp. 214–224.

[7] Chen, H., Wen, Y., Cheng, L., Kuang, S., Liu, Y., Li, W., Li, L., Zhang, R., Song, X., Li, W., et al. Autoos: make your os more powerful by exploiting large language models. In *Forty-first International Conference on Machine Learning* (2024).

[8] Chen, X. E., and Aamodt, T. M. Hybrid analytical modeling of pending cache hits, data prefetching, and mshrs. *ACM Transactions on Architecture and Code Optimization (TACO) 8*, 3 (2011), 1–28.

[9] Chung, I.-H., and Hollingsworth, J. K. Automated cluster-based web service performance tuning. In *Proceedings of the 13th IEEE International Symposium on High Performance Distributed Computing* (2004), pp. 36–44.

[10] Eyerman, S., Eeckhout, L., Karkhanis, T., and Smith, J. E. A mechanistic performance model for superscalar out-of-order processors. *ACM Transactions on Computer Systems (TOCS) 27*, 2 (2009), 1–37.

[11] Eyerman, S., Hoste, K., and Eeckhout, L. Mechanistic-empirical processor performance modeling for constructing cpi stacks on real hardware. In *(IEEE ISPASS) IEEE International Symposium on Performance Analysis of Systems and Software* (2011), IEEE, pp. 216–226.

[12] Ferdman, M., Adileh, A., Kocberber, O., Volos, S., Alisafaee, M., Jevdjic, D., Kaynak, C., Popescu, A. D., Ailamaki, A., and Falsafi, B. Clearing the clouds: A study of emerging scale-out workloads on modern hardware. In *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems* (2012), pp. 37–48.

[13] Gan, Y., Zhang, Y., Cheng, D., Shetty, A., Rathi, P., Katarki, N., Bruno, A., Hu, J., Ritchken, B., Jackson, B., Hu, K., Pancholi, M., He, Y., Clancy, B., Colen, C., Wen, F., Leung, C., Wang, S., Zaruvinsky, L., Espinosa, M., Lin, R., Liu, Z., Padilla, J., and Delimitrou, C. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (2019), pp. 3–18.

[14] Garre, C., Mundo, D., Gubitosa, M., and Toso, A. Performance comparison of real-time and general-purpose operating systems in parallel physical simulation with high computational cost. Tech. rep., SAE Technical Paper, 2014.

[15] Hartstein, A., and Puzak, T. R. The optimum pipeline depth for a microprocessor. *ACM Sigarch Computer Architecture News 30*, 2 (2002), 7–13.

[16] Hatt, N., Sivitz, A., and Kuperman, B. A. Benchmarking operating systems. In *Conference for Undergraduate Research in Computer Science and Mathematics* (2007), p. 121.

[17] Hwang, K., Bai, X., Shi, Y., Li, M., Chen, W.-G., and Wu, Y. Cloud performance modeling with benchmark evaluation of elastic scaling strategies. *IEEE Transactions on parallel and distributed systems 27*, 1 (2015), 130–143.

[18] Infantolino, J. K., Park, S., and Shires, D. R. Selecting a benchmark suite to profile high-performance computing (hpc) machines. Tech. rep., U.S. Army Research Laboratory, nov 2014.

[19] Ipek, E., De Supinski, B. R., Schulz, M., and McKee, S. A. An approach to performance prediction for parallel applications. In *European Conference on Parallel Processing* (2005), Springer, pp. 196–205.

[20] Îpek, E., McKee, S. A., Caruana, R., de Supinski, B. R., and Schulz, M. Efficiently exploring architectural design spaces via predictive modeling. *ACM SIGOPS Operating Systems Review 40*, 5 (2006), 195–206.

[21] Kanoun, K., and Crouzet, Y. Dependability benchmarks for operating systems. *International Journal of Performability Engineering 2*, 3 (2006), 275–287.

[22] Koopman, P., Sung, J., Dingman, C., Siewiorek, D., and Marz, T. Comparing operating systems using robustness benchmarks. In *Proceedings of SRDS'97: 16th IEEE Symposium on Reliable Distributed Systems* (1997), IEEE, pp. 72–79.

[23] Kopytov, A. Sysbench: a system performance benchmark. *http://sysbench. sourceforge. net/* (2004).

[24] Lee, B. C., and Brooks, D. M. Accurate and efficient regression modeling for microarchitectural performance and power prediction. *ACM SIGOPS operating systems review 40*, 5 (2006), 185–194.

[25] Lenk, A., Menzel, M., Lipsky, J., Tai, S., and Offermann, P. What are you paying for? performance benchmarking for infrastructure-as-a-service offerings. In *2011 IEEE 4th International Conference on Cloud Computing* (2011), IEEE, pp. 484–491.

[26] Marieska, M. D., Hariyanto, P. G., Fauzan, M. F., Kistijantoro, A. I., and Manaf, A. On performance of kernel based and embedded real-time operating system: Benchmarking and analysis. In *2011 International Conference on Advanced Computer Science and Information Systems* (2011), IEEE, pp. 401–406.

[27] Mittal, V., Bruel, P., Milojicic, D., and Frachtenberg, E. Adaptive stopping rule for performance measurements. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (2023), pp. 1288–1297.

[28] Moschetta, J., and Casale, G. Ofbench: An enterprise application benchmark for cloud resource management studies. In *2012 14th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing* (2012), IEEE, pp. 393–399.

[29] openEuler A-Tune Project. A-Tune: An ai-based operating system performance tuning engine. https://github.com/openeuler-mirror/A-Tune/blob/master/README-zh.md, 2026. README-zh.md, GitHub repository.

[30] Phoronix Media. Phoronix Test Suite: Linux Testing and Benchmarking Platform, Automated Testing, Open-Source Benchmarking. https://www.phoronix-test-suite.com/, 2026. Accessed: 2026-05-19.

[31] Saavedra, R. H., and Smith, A. J. Analysis of benchmark characteristics and benchmark performance prediction. *ACM Transactions on Computer Systems 14*, 4 (1996), 344–384.

[32] Sharkawi, S., Desota, D., Panda, R., Indukuru, R., Stevens, S., Taylor, V., and Wu, X. Performance projection of hpc applications using spec cfp2006 benchmarks. In *2009 IEEE International Symposium on Parallel & Distributed Processing* (2009), IEEE, pp. 1–12.

[33] Silva, M., Cerdeira, D., Pinto, S., and Gomes, T. Operating systems for internet of things low-end devices: Analysis and benchmarking. *IEEE Internet of Things Journal 6*, 6 (2019), 10375–10383.

[34] Smith, B., Grehan, R., Yager, T., Niemi, D., and Voellm, A. Byte-unixbench: A unix benchmark suite. *Technical report* (2011).

[35] Sun, Y., Bi, F., He, J., Qu, S., Wu, Y., and Hou, P. Kconfigtune: Automatic performance tuning for linux kernel configuration. *IEEE Transactions on Computers* (2025).

[36] Tousi, A., and Luján, M. Comparative analysis of machine learning models for performance prediction of the spec benchmarks. *IEEE Access 10* (2022), 11994–12011.

[37] Van den Steen, S., De Pestel, S., Mechri, M., Eyerman, S., Carlson, T., Black-Schaffer, D., Hagersten, E., and Eeckhout, L. Micro-architecture independent analytical processor performance and power modeling. In *2015 IEEE international symposium on performance analysis of systems and software (ISPASS)* (2015), IEEE, pp. 32–41.

[38] Venkataraman, S., Yang, Z., Franklin, M. J., Recht, B., and Stoica, I. Ernest: Efficient performance prediction for large-scale advanced analytics. In *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation* (2016).

[39] Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., and Krcmar, H. Wessbas: Extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. *Software & Systems Modeling 17*, 2 (2018), 443–477.

[40] Wang, Q., Wang, R., Hu, Y., Shi, X., Liu, Z., Ma, T., Song, H., and Shi, H. Keentune: Automated tuning tool for cloud application performance testing and optimization. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (2023), pp. 1487–1490.

[41] Wang, Y., Lee, V., Wei, G.-Y., and Brooks, D. Predicting new workload or cpu performance by analyzing public datasets. *ACM Transactions on Architecture and Code Optimization (TACO) 15*, 4 (2019), 1–21.

[42] Watfa, M. K., and Moubarak, M. A benchmarking tool for wireless sensor network embedded operating systems. *J. Networks 9*, 8 (2014), 1971–1984.

[43] Zheng, X., John, L. K., and Gerstlauer, A. Accurate phase-level cross-platform power and performance estimation. In *Proceedings of the 53rd Annual Design Automation Conference* (2016), pp. 1–6.

A Feature Collection Inventory

Table 16 lists representative pre-run context and in-run probe signals.

B Artifact and Transfer Notes

The anonymous bundle contains raw CPU/UnixBench records, source, saved models, HTTP logs, and audited split/output JSON. It includes the M1 router–reconstructor grid, per-run feature ablations, waveforms, and scripts for the HTTP, transfer, and feature-use analyses. The model grid and feature ablations reuse the five-run session; complete command provenance is unavailable for some legacy tables.

The strict CloudSuite split audit is the session-audit JSON under the round-8 results directory; its input hash prefix is `ff3bc3c19b75` (full SHA256 is stored in the artifact). Session 5 also contains paired per-cell phase timings: probe/full totals are 790.86/1,119.02 s for FPM4 and 795.10/1,094.00 s for FPM8 across five loads. Sessions 6, 8, 9, and 14 contain the four valid exploratory FPM4 accuracy/cost pairs summarized in Table 7; sessions 9 and 14 use matched 60-second cells and full-first order. Session 10 has timing provenance (probe/full 479.122/787.639 s) but its full-scale20 operation mix is invalid and it is excluded from accuracy replication. All valid sessions use the pre-fixed evenly spaced interpolation policy with probe loads 20/60/100 and full-reference loads 20/40/60/80/100. Session 9 records are under `round-8-cloudsuite/results/bsr8-local-supplemental-session9-20261014T030226Z-1034987`; session 10 records are under `round-8-cloudsuite/results/bsr8-local-supplemental-session10-20261014T032440Z-1155738`; session 14 records are under `round-8-cloudsuite/results/bsr8-local-supplemental-session14-20261014T054819Z-2356052`. Timings include fresh service reset/setup per cell and are not selective single-deployment end-to-end runs. Partial FPM8 session-6 and session-7 provenance is retained but excluded. The artifact includes raw records and the omitted-component audit with exact run and fold joins.

Table 16: Purpose-oriented feature groups organized by collection stage. Pre-run signals form the system context $\mathbf{x}_i$, while in-run probe signals form the probe vector $\mathbf{p}_{i,j}$.

Stage	Signal type	Purpose	Representative signals
Pre-run	Static host profile	Identify stable host configuration.	CPU topology, NUMA layout, memory size, gcc versions
	System pressure	Capture transient pre-run load.	CPU/GPU utilization, iowait, load average, runnable tasks
	PMU / perf	Profile microarchitectural behavior.	IPC, cycles/s, instructions/s, cache/branch miss ratios
In-run	Probe progress	Record observed benchmark progress.	Configured probing duration, elapsed wall-clock time
	eBPF-based tracing	Observe prefix-level kernel activity.	Scheduling-switch rate, syscall-entry rate, eBPF availability
	Resource usage	Summarize prefix resource usage.	User/system/idle CPU ratio, minor faults, major faults
	Workload identity	Provide workload semantic context.	CPU, memory, I/O, thread, syscall, GPU category
	Probe flags	Mark probe mode and failures.	Micro workload flag, real-component flag, timeout flag